# Exercise 4: MQTT Publish/Subscribe

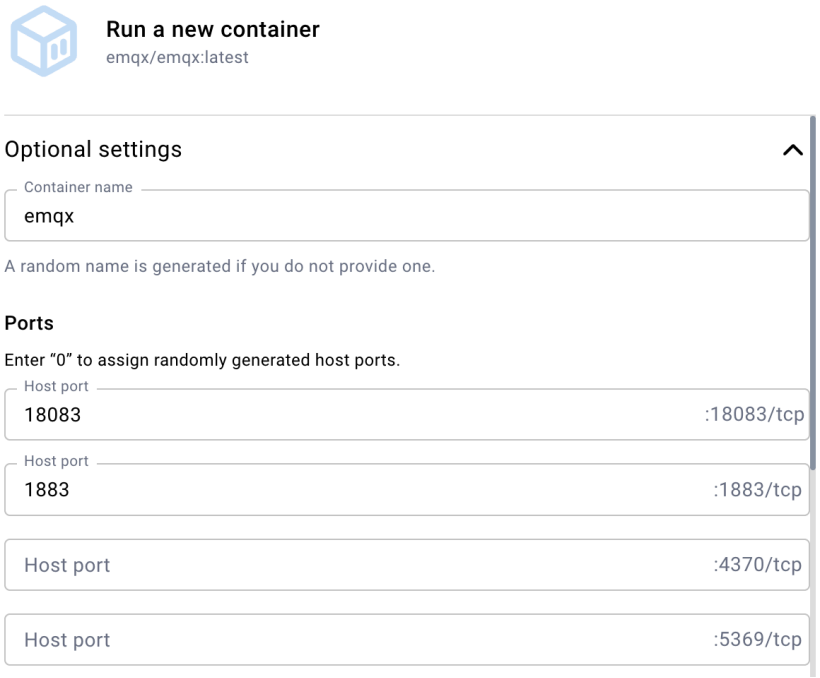
**Week:** 4 (Hands-On Laboratory Exercise)

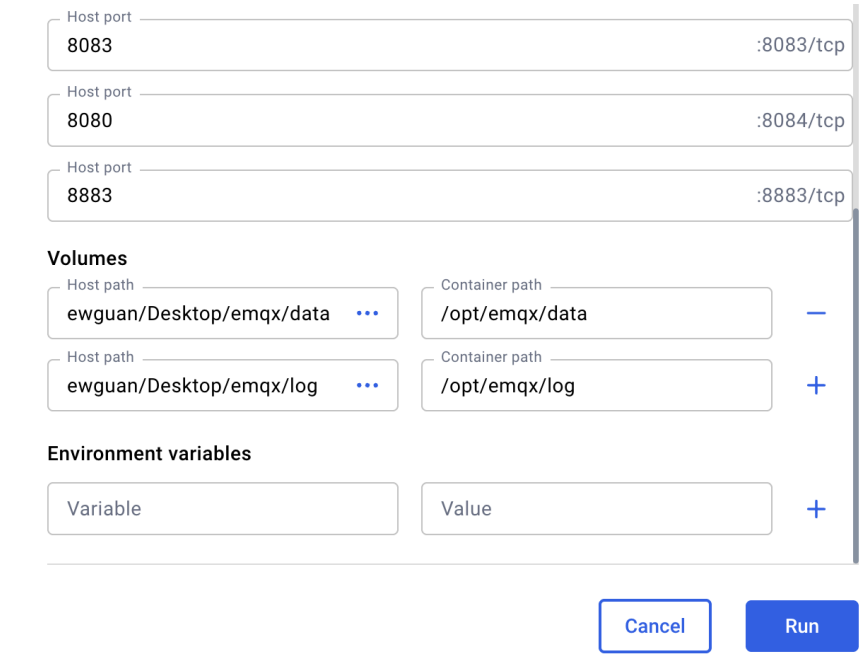
**Objective:** To deploy an enterprise-grade MQTT broker (EMQX) locally via Docker, configure Node-RED as an automated MQTT publishing client, utilize Postman as an MQTT subscribing client, and explore topic hierarchies using wildcards.

---

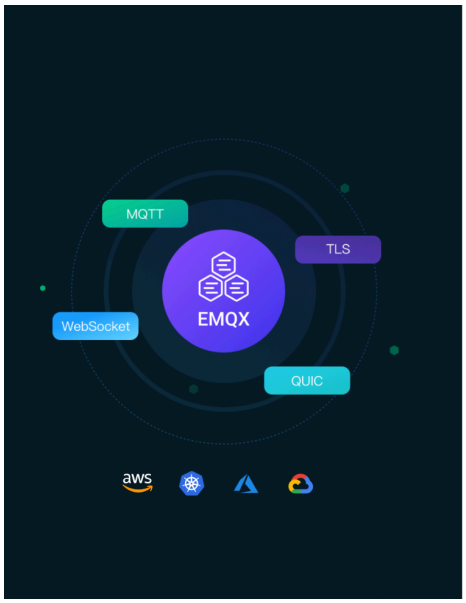
## Step 1: Initialize the EMQX MQTT Broker via Docker

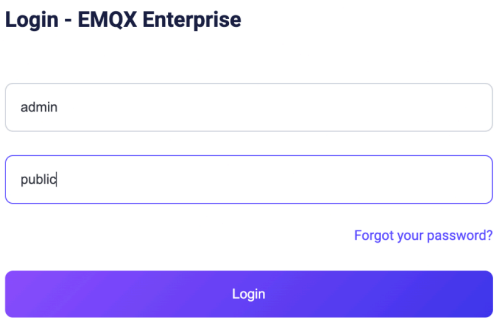
1. **Open Docker Desktop:** Ensure the Docker daemon is running in the background.
2. **Launch via Docker Desktop:** Open your computer's Docker Desktop click the run button for the EMQX image, and use the settings as follow:
    - **Ports***:* 18083, 1883, 8083, 8080, 8883
    - **Volumes***:*
        - /opt/emqx/data: This is the most critical folder. It contains the Mnesia database, which stores:
        - /opt/emqx/log: Highly recommended. This folder stores all operational logs, which are essential for troubleshooting.






3. **Verify the Broker:** Open your web browser and navigate to http://localhost:18083. Log in with the default credentials (admin / public) to verify the broker is active.

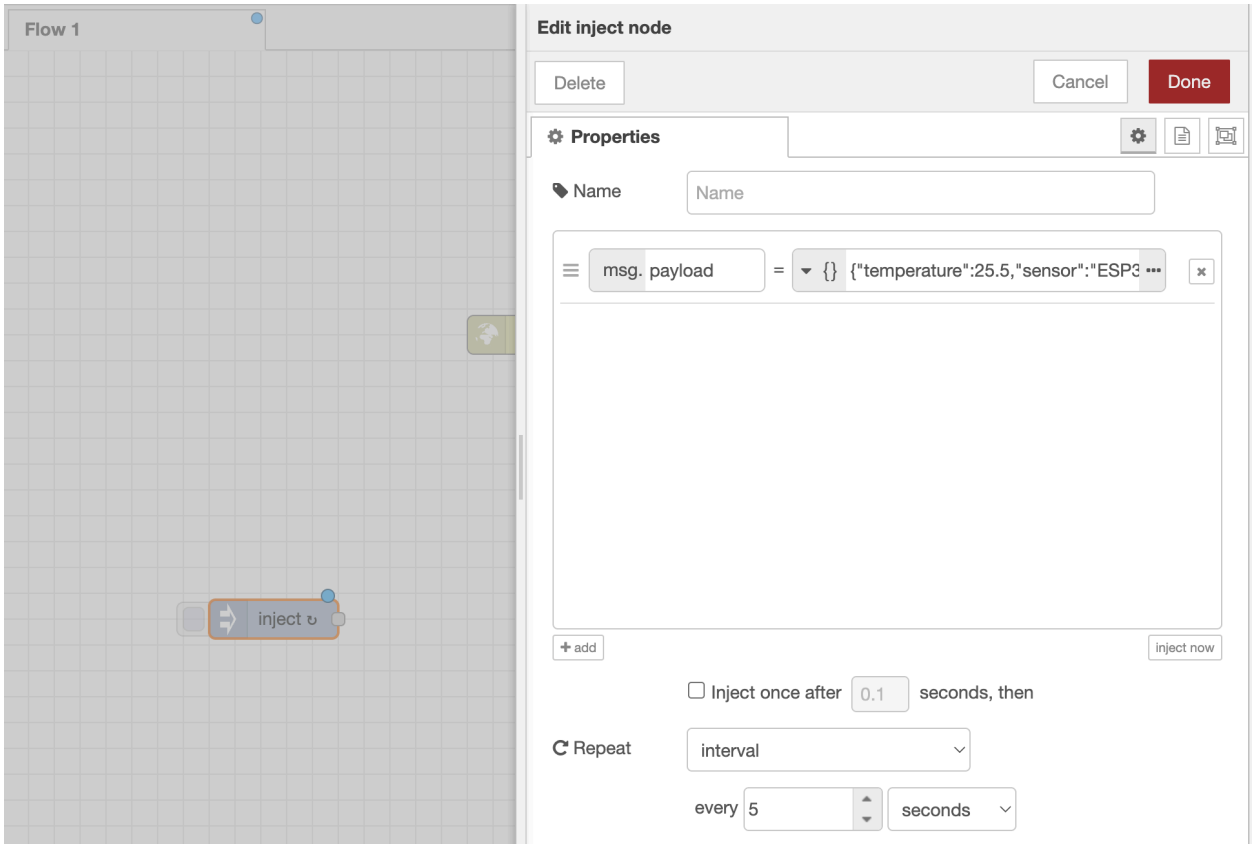


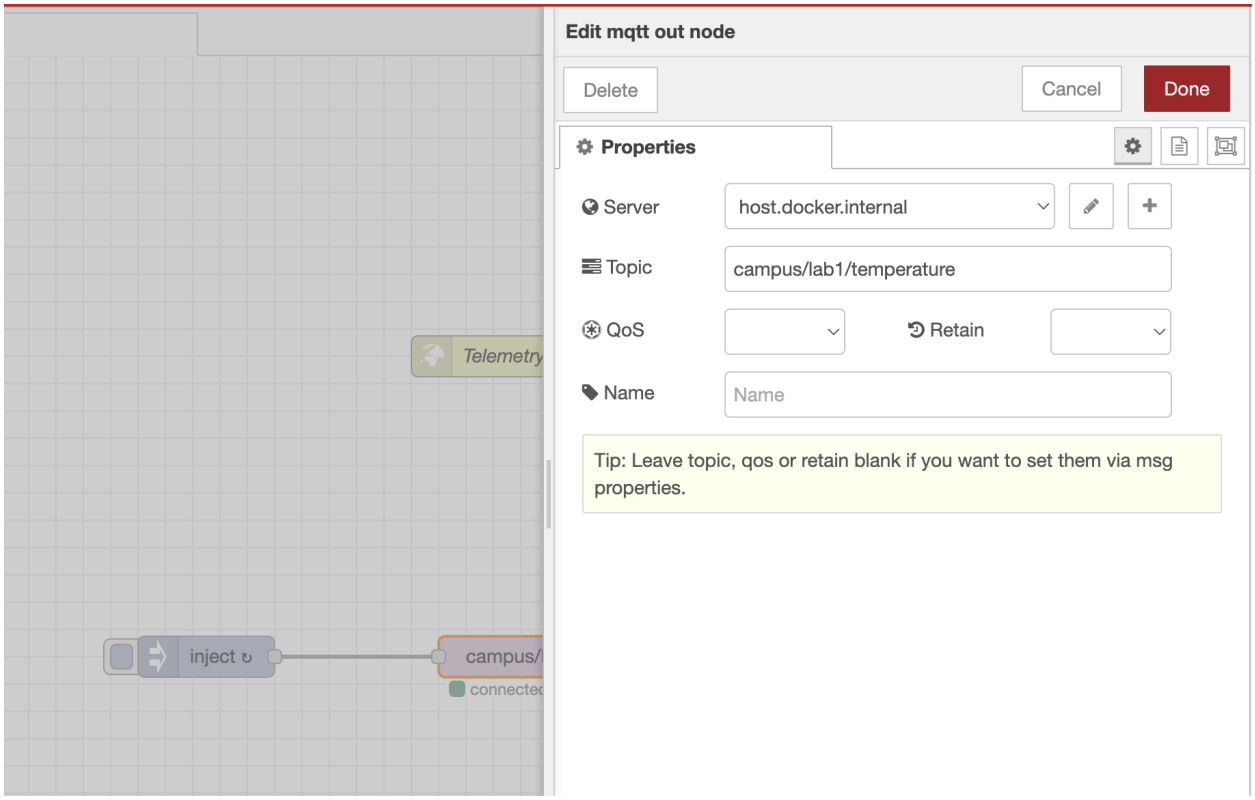
# Step 2: Configure Node-RED as an MQTT Publisher

Now that the broker is listening, we will configure Node-RED to act as an IoT edge device (Client), periodically publishing simulated telemetry data.

1. **Add an Inject Node:** Drag an inject node onto your Node-RED workspace. Double-click it:
    - **Payload:** Change to JSON and enter: {"temperature": 25.5, "sensor": "ESP32_01"}
    - **Repeat:** Set to interval every 5 seconds.




2. **Add an MQTT Out Node:** Drag an mqtt out node onto the workspace and wire it to the inject node. Double-click it:
    - **Server:** Click the pencil icon to add a new MQTT broker. Set the Server to host.docker.internal and Port to 1883. Click Add.
    - **Topic:** Type campus/lab1/temperature.



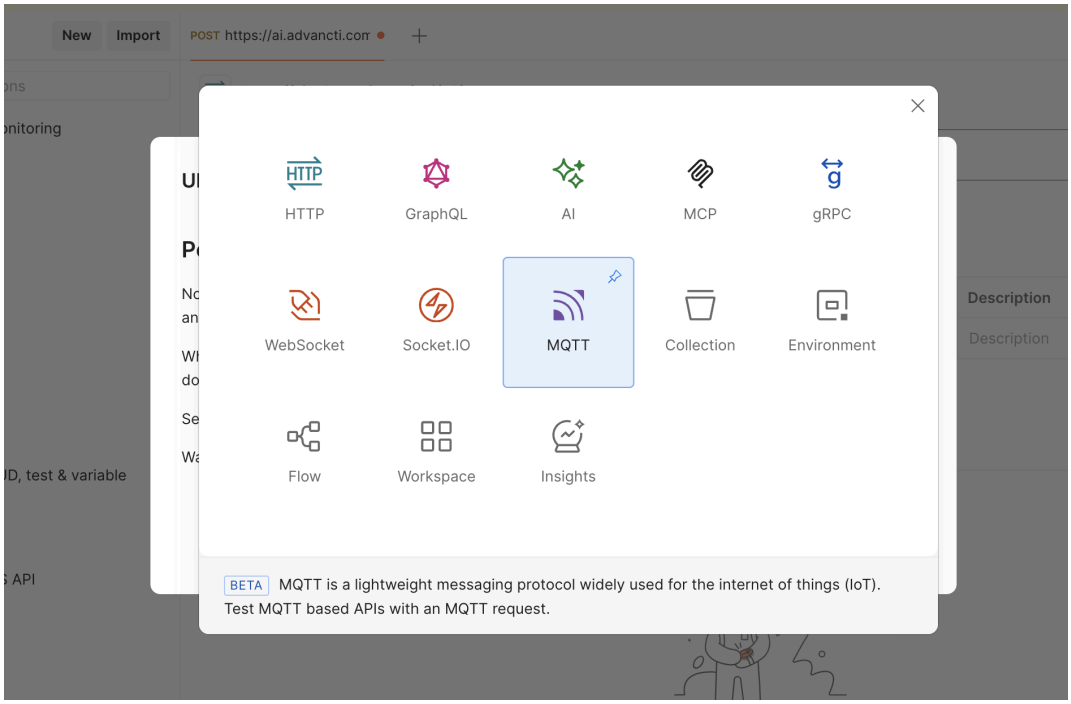

3. **Deploy:** Click **Deploy**. Your Node-RED client is now silently publishing a payload to the broker every 5 seconds.

---

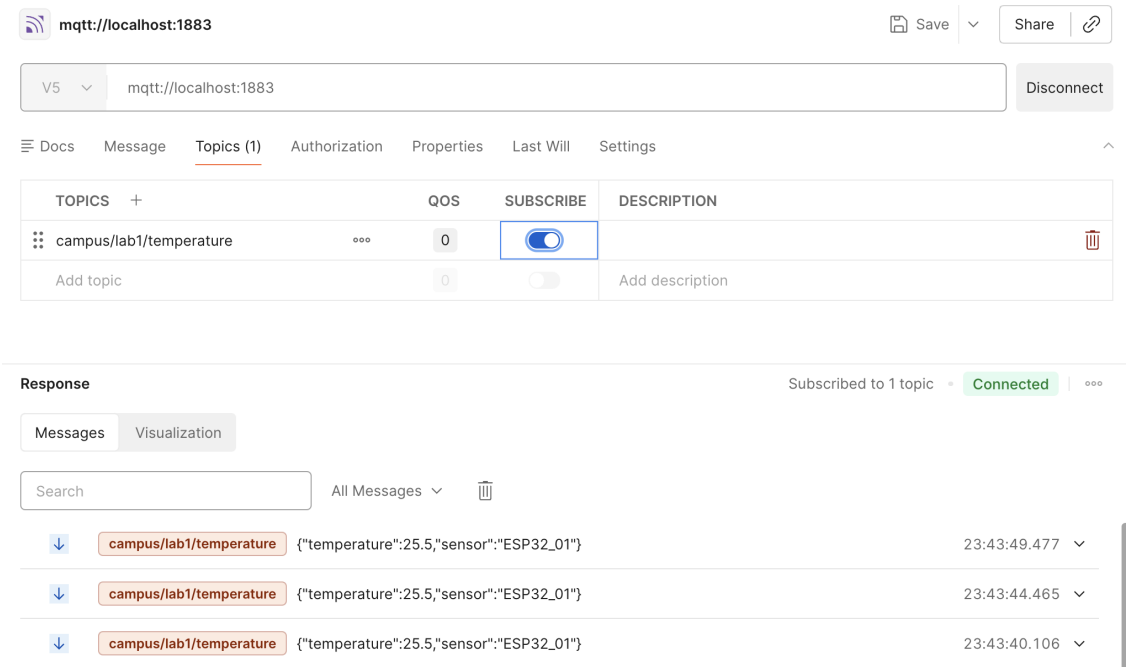
# Step 3: Configure Postman as an MQTT Subscriber

Data published to a broker vanishes if no one is listening! We will use Postman to act as a second MQTT client, subscribing to the topic to catch the incoming telemetry.

1. **Open Postman for MQTT:** In Postman, click **New** and select **MQTT** (instead of standard HTTP).




2. **Connect to the Broker:** * Enter the broker URL: mqtt://localhost:1883.
    - Click **Connect**. You should see a green "Connected" status.
3. **Subscribe to the Topic:**
    - In the "Topics" or "Subscribe" tab, enter the exact topic Node-RED is publishing to: campus/lab1/temperature.
    - Click **Subscribe**.



4. **Observe the Data:** Look at the message stream window in Postman. Every 5 seconds, a new JSON payload should appear in real-time, pushed automatically by the EMQX broker!

# Part 4: Advanced Topic Hierarchies and Wildcards

MQTT's true power lies in topic hierarchies. You will now modify your Node-RED publisher and use Postman to experiment with the multi-level (#) and single-level (+) wildcards.

**Execution:**

1. **Modify Node-RED Publishers:** Create three separate inject -> mqtt out flows in Node-RED. Set them to inject every 5 seconds, publishing to these three distinct topics:
    - campus/lab1/temperature
    - campus/lab2/temperature
    - campus/lab1/humidity
2. **Experiment with the # (Multi-Level) Wildcard:**
    - In Postman, unsubscribe from the previous topic and subscribe to: campus/#
    - *Observation:* You should now receive messages from *all three* topics. The # wildcard matches any number of levels after campus/.
3. **Experiment with the + (Single-Level) Wildcard:**
    - Unsubscribe, and now subscribe to: campus/+/temperature
    - *Observation:* You should receive messages from lab1 and lab2 temperature topics, but *not* the humidity topic. The + wildcard substitutes exactly one level in the hierarchy!

# Part 5: Discussion & Architectural Analysis

Based on the laboratory exercises you just completed, answer the following questions to solidify your understanding of event-driven architectures.

**Question 1: The Role of the Broker**

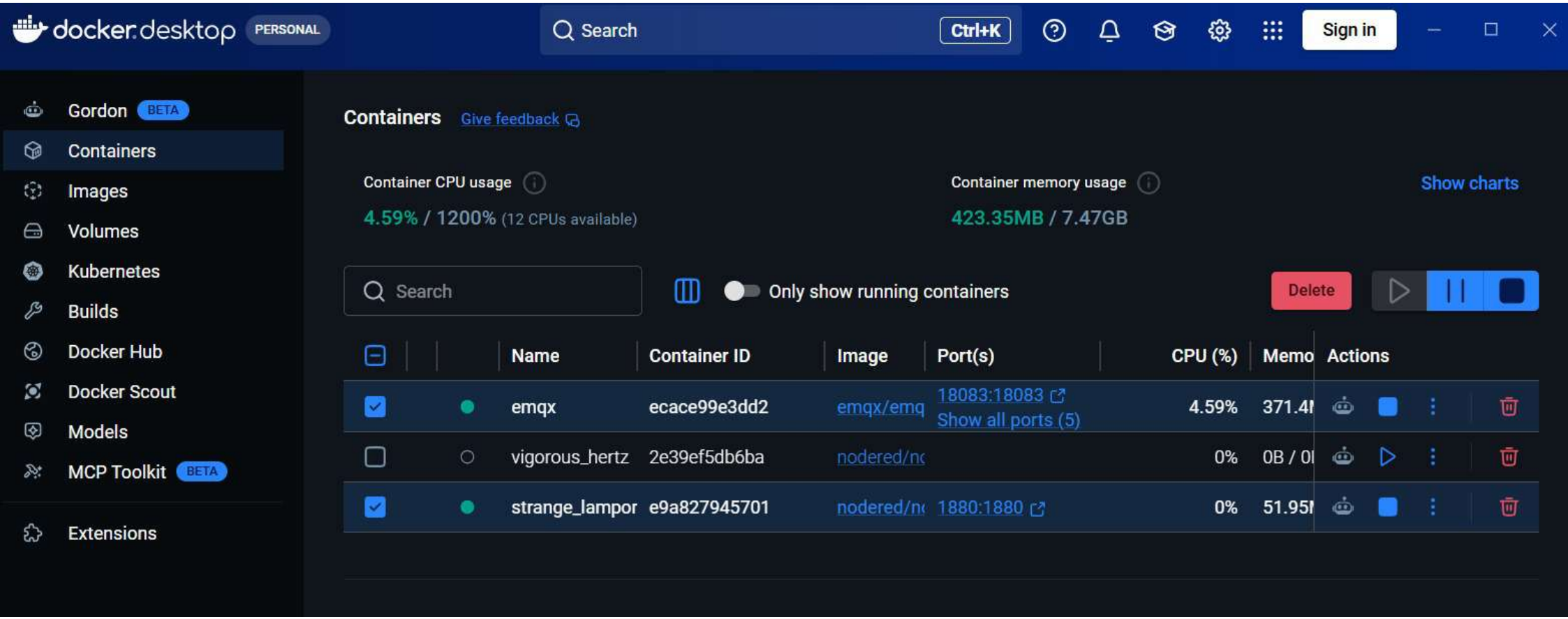
In Week 3, you sent data directly from Postman (Client) to Node-RED (Server) using HTTP. Today, both Node-RED and Postman connected to the EMQX Broker instead. Explain the architectural role of the MQTT broker. What is the advantage of MQTT compared to the HTTP protocol?

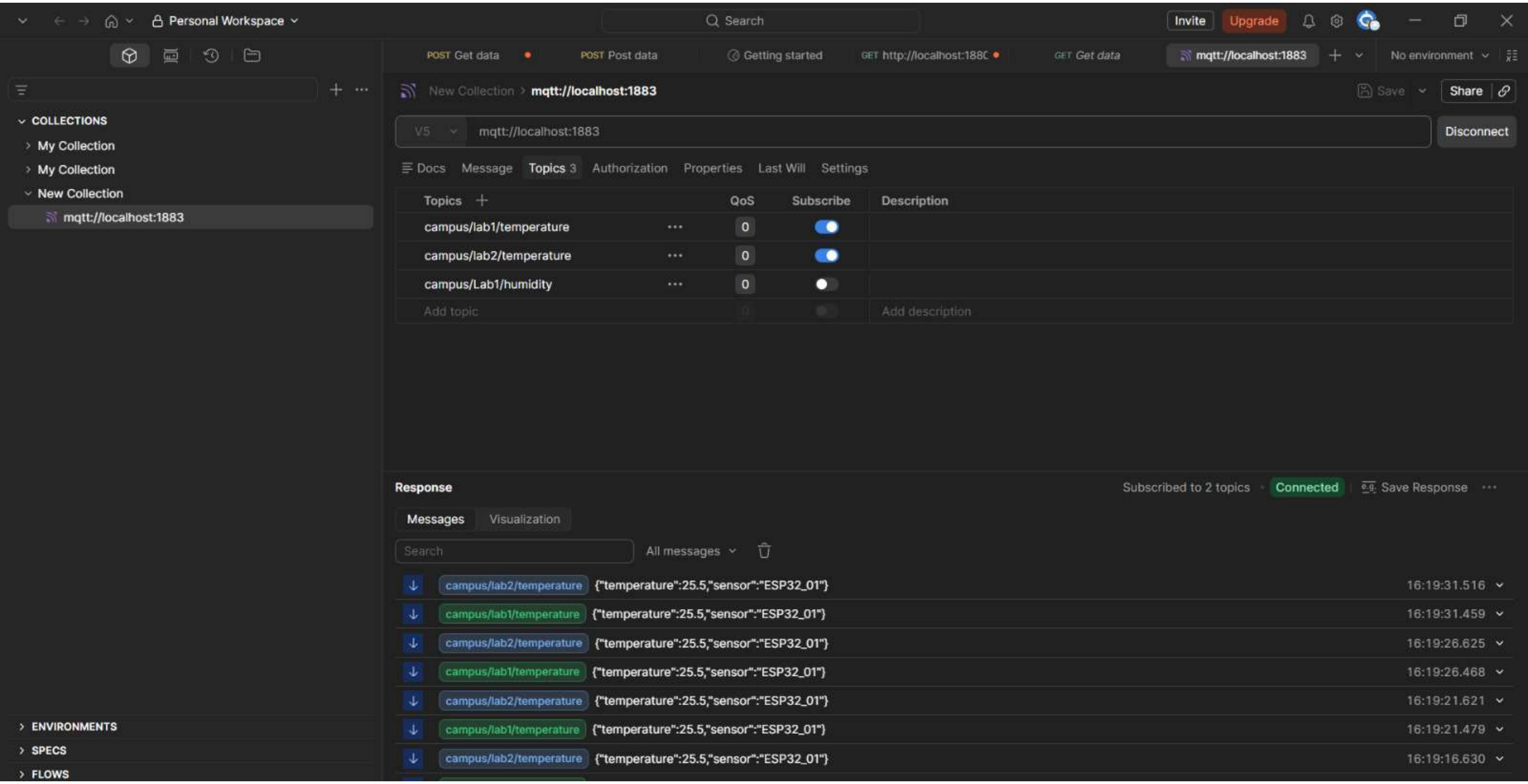
**Question 2: Publish vs. Subscribe**

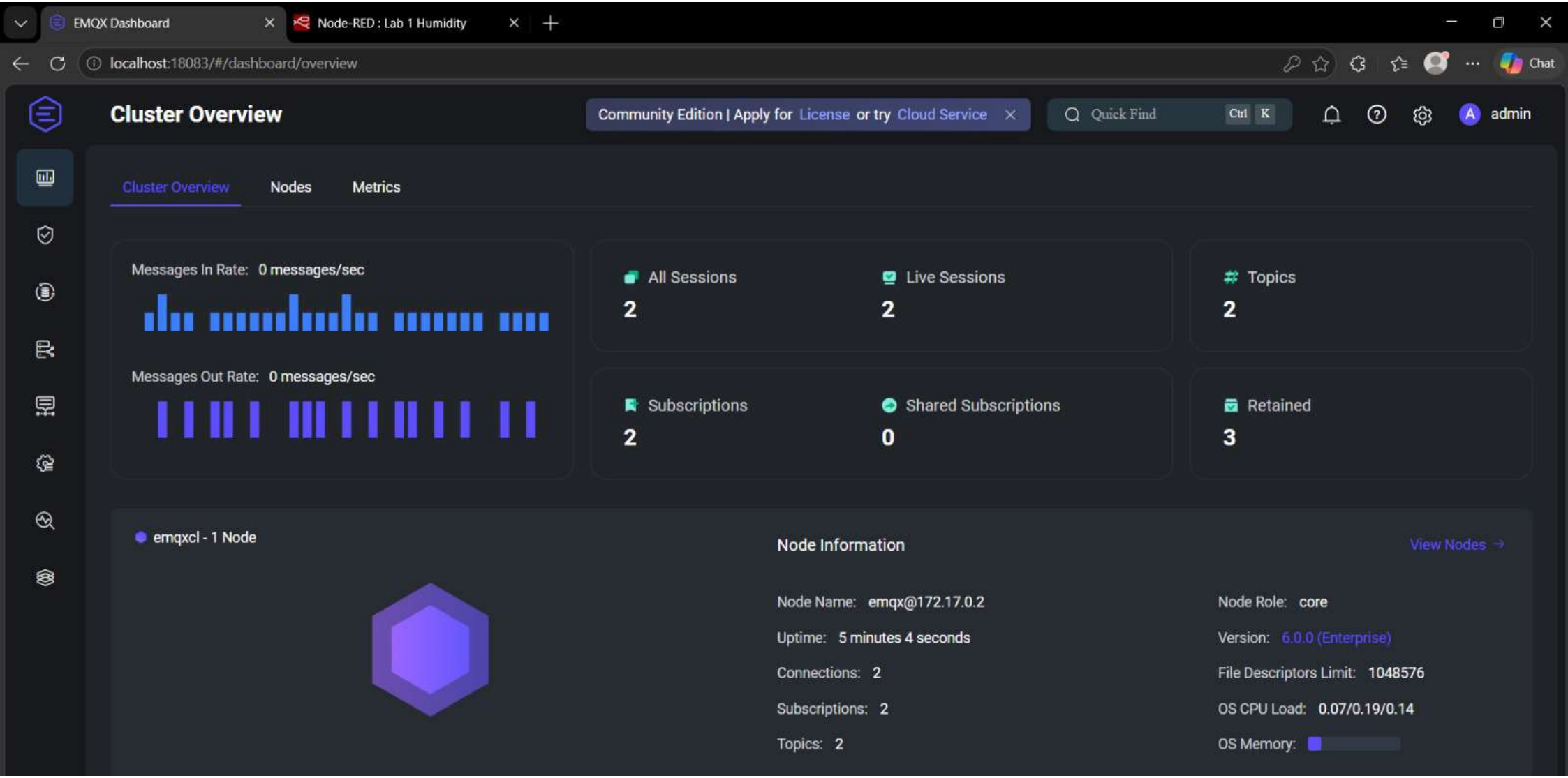
Define the specific roles of a "Publisher" and a "Subscriber" in the MQTT protocol. Can a single IoT device (like an ESP32) be both a publisher and a subscriber simultaneously? Provide a real-world example of why a device would need to do both.

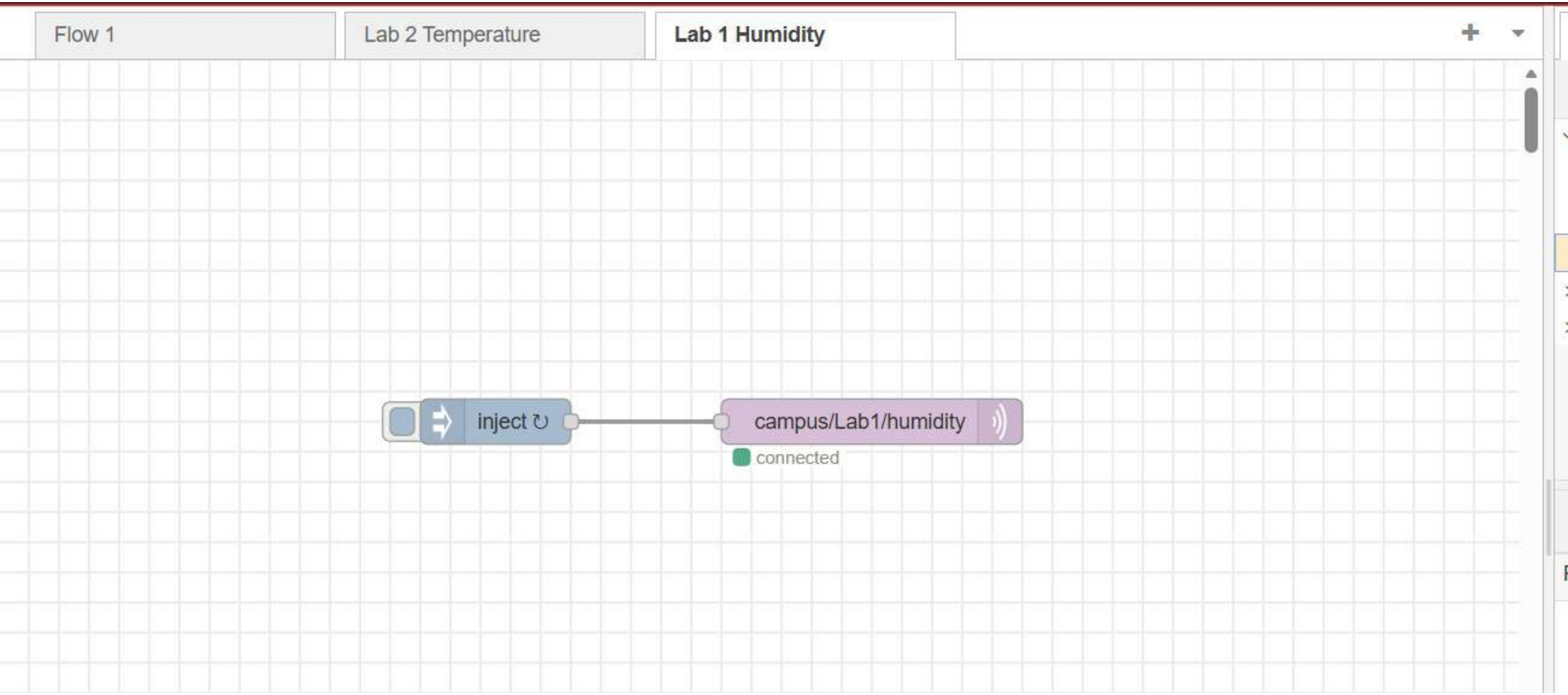
**Question 3: Client vs. Server**

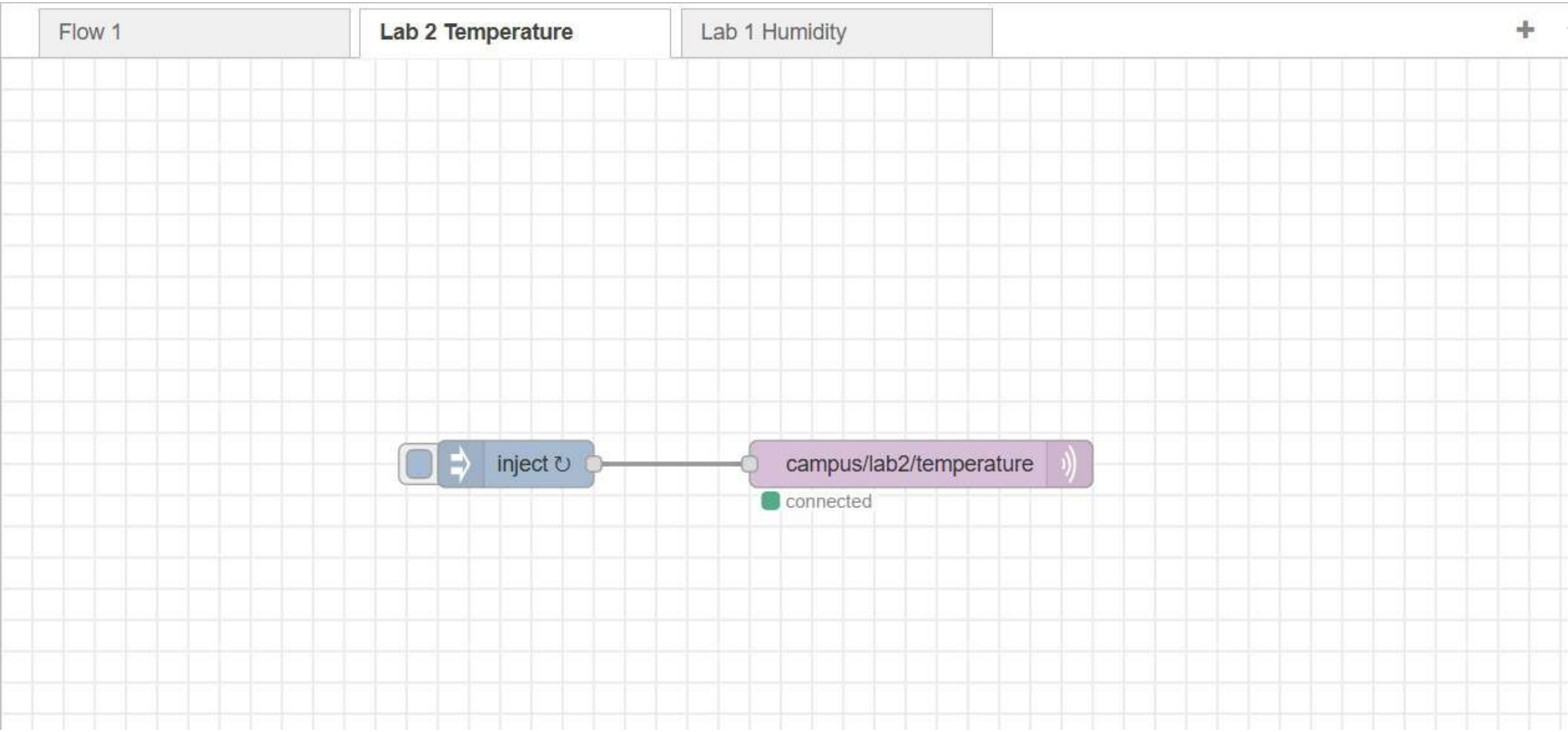
In an HTTP REST API, the server must be running and listening before the client can send a POST request. In MQTT, what happens if Postman (the subscribing client) disconnects for 10 minutes while Node-RED (the publishing client) continues to send data? How does the broker handle this?

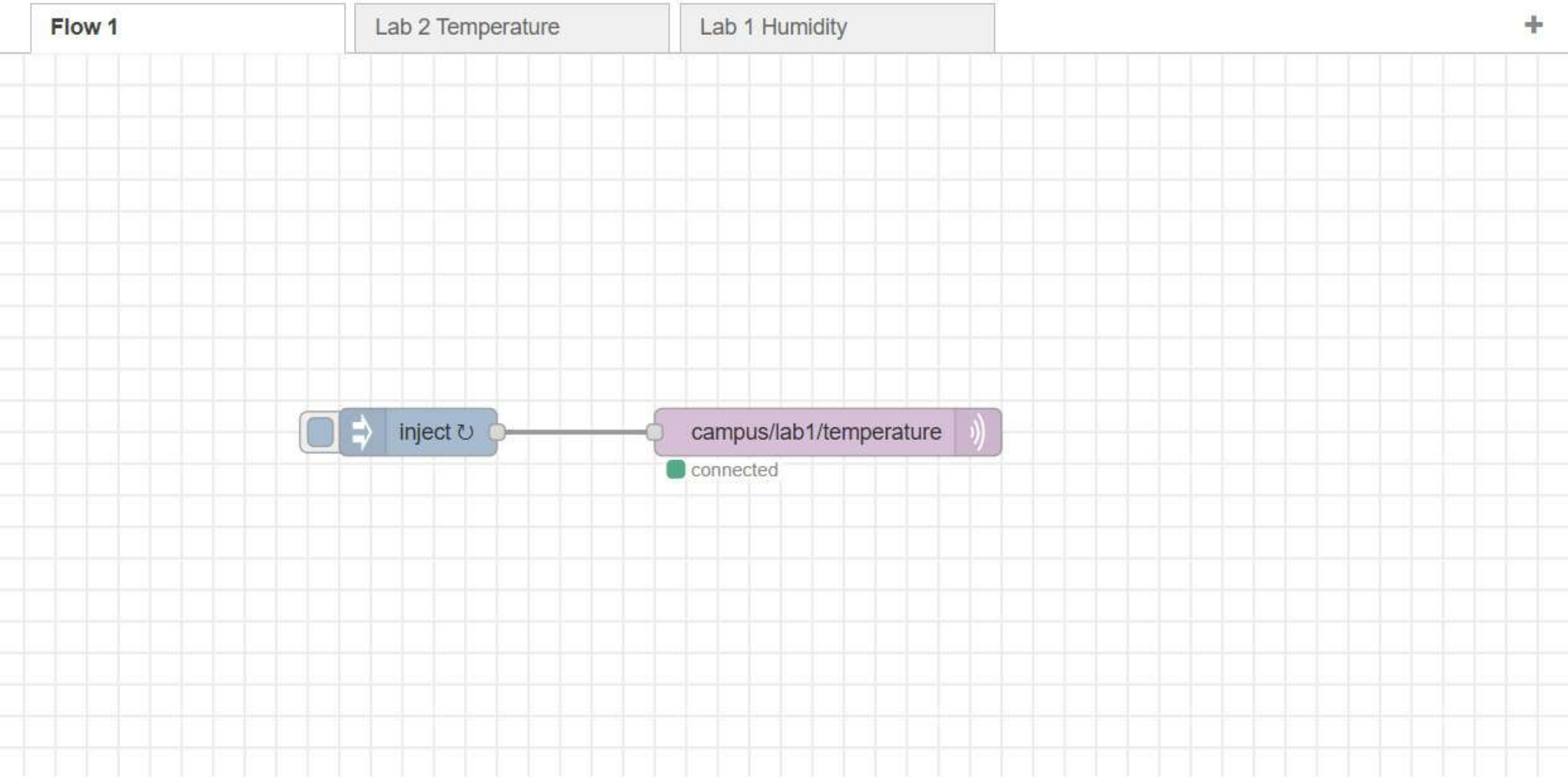

docker desktop
PERSONAL
Search
Ctrl+K
Sign in
Gordon
BETA
Containers
Images
Volumes
Kubernetes
Builds
Docker Hub
Docker Scout
Models
MCP Toolkit
BETA
Extensions
Containers
Give feedback
Container CPU usage
4.59% / 1200% (12 CPUs available)
Container memory usage
423.35MB / 7.47GB
Show charts
Search
Only show running containers
Delete
Name
Container ID
Image
Port(s)
CPU (%)
Memo
Actions
emqx
ecace99e3dd2
emqx/emq
18083:18083
Show all ports (5)
4.59%
371.4
vigorous_hertz
2e39ef5db6ba
nodered/nc
0%
0B / 0
strange_lampor
e9a827945701
nodered/nc
1880:1880
0%
51.95

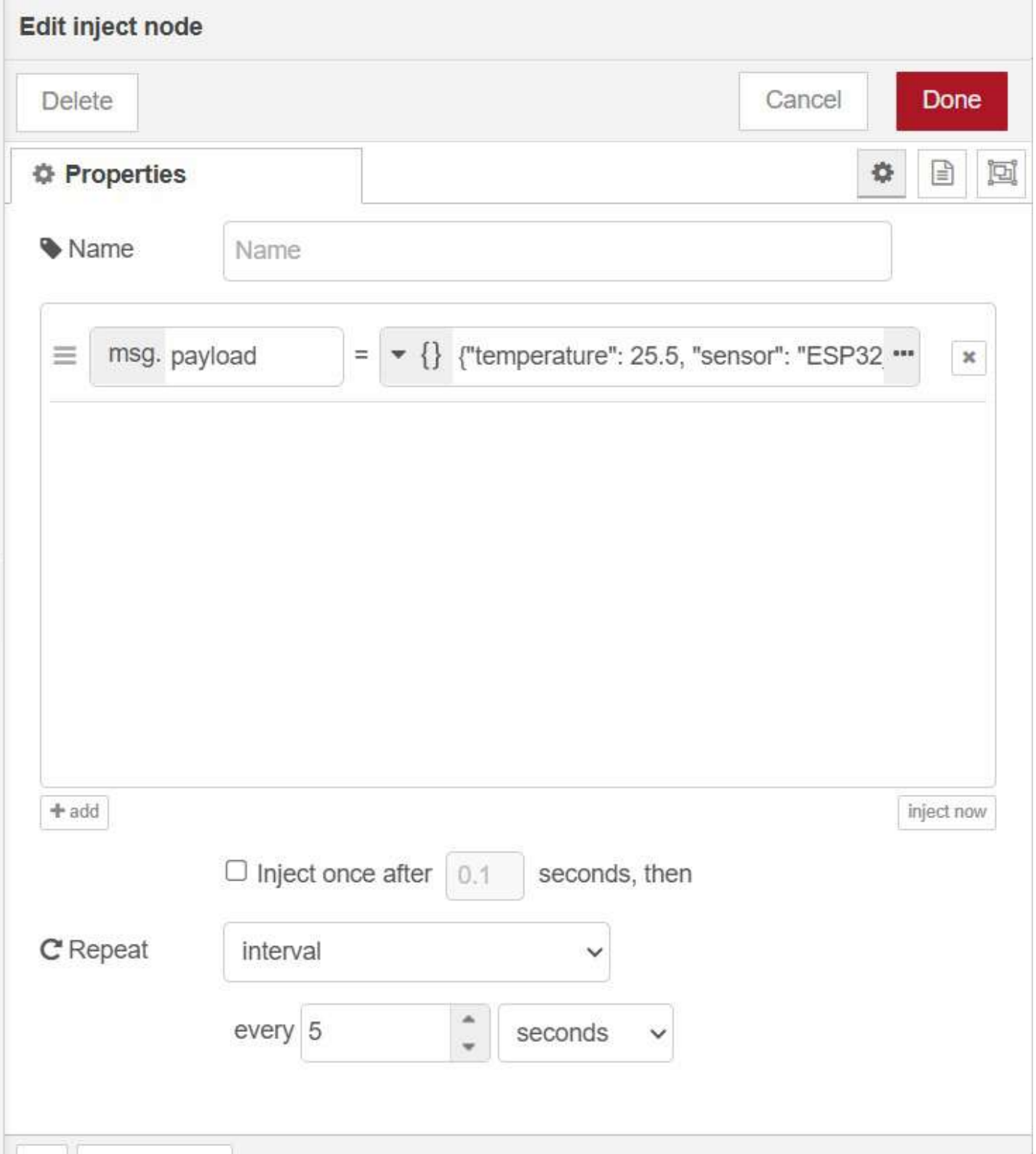

Personal Workspace
Search
Invite
Upgrade
POST Get data
POST Post data
Getting started
GET http://localhost:1880
GET Get data
mqtt://localhost:1883
No environment
COLLECTIONS
My Collection
My Collection
New Collection
mqtt://localhost:1883
New Collection > mqtt://localhost:1883
Save
Share
V5
mqtt://localhost:1883
Disconnect
Docs
Message
Topics 3
Authorization
Properties
Last Will
Settings
Topics
QoS
Subscribe
Description
campus/lab1/temperature
0
campus/lab2/temperature
0
campus/Lab1/humidity
0
Add topic
Add description
Response
Subscribed to 2 topics
Connected
Save Response
Messages
Visualization
Search
All messages
campus/lab2/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:31.516
campus/lab1/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:31.459
campus/lab2/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:26.625
campus/lab1/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:26.468
campus/lab2/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:21.621
campus/lab1/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:21.479
campus/lab2/temperature {"temperature":25.5,"sensor":"ESP32_01"} 16:19:16.630
ENVIRONMENTS
SPECS
FLOWS


EMQX Dashboard
Node-RED : Lab 1 Humidity
localhost:18083/#/dashboard/overview
Chat
Cluster Overview
Community Edition | Apply for License or try Cloud Service
Quick Find
Ctrl K
admin
Cluster Overview
Nodes
Metrics
Messages In Rate: 0 messages/sec
Messages Out Rate: 0 messages/sec
All Sessions
2
Live Sessions
2
Topics
2
Subscriptions
2
Shared Subscriptions
0
Retained
3
emqxcl - 1 Node
Node Information
View Nodes
Node Name: emqx@172.17.0.2
Uptime: 5 minutes 4 seconds
Connections: 2
Subscriptions: 2
Topics: 2
Node Role: core
Version: 6.0.0 (Enterprise)
File Descriptors Limit: 1048576
OS CPU Load: 0.07/0.19/0.14
OS Memory:


Flow 1
Lab 2 Temperature
Lab 1 Humidity
inject
campus/Lab1/humidity
connected


Flow 1
Lab 2 Temperature
Lab 1 Humidity
inject
campus/lab2/temperature
connected


Flow 1
Lab 2 Temperature
Lab 1 Humidity
inject
campus/lab1/temperature
connected


Edit inject node
Delete
Cancel
Done
Properties
Name
Name
msg. payload
=
{}
{"temperature": 25.5, "sensor": "ESP32
add
inject now
Inject once after
0.1
seconds, then
Repeat
interval
every
5
seconds

**Part 5: Discussion & Architectural Analysis**

**1. The Role of MQTT Broker (Peranan Broker)**

- **Penjelasan:** Dalam *MQTT architecture*, Broker bertindak sebagai pusat perantara atau ***central hub*** untuk semua komunikasi. Berbeza dengan protokol HTTP (Week 3) yang bersifat ***request-response***, MQTT menggunakan model ***publish-subscribe***. *Devices* tidak perlu tahu *IP address* masing-masing; mereka hanya perlu *connect* ke Broker untuk *transmit* atau *receive data*.
- **Advantage of MQTT:**
    - **Bandwidth Efficiency:** Mempunyai ***packet overhead*** yang sangat kecil, sesuai untuk *low-resource devices*.
    - **Power Saving:** *Devices* tak perlu sentiasa buat ***polling***, jadi sangat menjimatkan bateri.
- **Contoh Kehidupan Seharian (Analogi Shopee):** Broker macam **Gudang Shopee**. Penjual (Publisher) hantar barang ke gudang tanpa perlu tahu siapa pembeli. Gudang (Broker) pula akan agihkan barang kepada pembeli (Subscriber) yang betul. Tanpa gudang, penjual kena hantar satu-satu barang ke setiap rumah pembeli (macam HTTP), yang mana sangat memenatkan.

---

**2. Publisher vs. Subscriber**

- **Definition:**
    - **Publisher:** *Device* yang menghantar data ke broker mengikut ***specific topic*** (Contoh: Node-RED hantar data suhu).
    - **Subscriber:** *Device* yang melanggan topik untuk terima data (Contoh: Postman sebagai pemantau).
- **Device Capability:** Ya, satu peranti (macam ESP32) boleh jadi ***publisher and subscriber simultaneously*** (serentak).
- **Contoh Kehidupan Seharian (Analogi WhatsApp Group):** Dalam *Group WhatsApp*, bila kau taip dan hantar mesej, kau adalah **Publisher**. Bila kau terima mesej dari kawan lain, kau adalah **Subscriber**. Kau buat dua-dua kerja ni dalam satu peranti yang sama.

---

**3. Client vs. Server Connection**

- **Scenario Analysis (Jika Disconnect 10 Minit):** Dalam *basic MQTT*, jika Postman (*subscriber*) terputus sambungan (*disconnected*) semasa Node-RED terus hantar data, data tersebut biasanya akan **hilang** kerana MQTT secara *default* adalah ***real-time***.
- **Broker Handling Mechanism:**
    1. **Retained Messages:** Broker simpan *last known message*. Bila *client connect* balik, dia terus dapat info paling *latest*.
    2. **Persistent Sessions:** Broker tolong simpan kejap mesej yang terlepas dan hantar semula bila *client* tadi *online*.
- **Contoh Kehidupan Seharian (Analogi Live Football):** Bayangkan kau tengok bola ***live*** tapi internet *down* 10 minit. Kau akan terlepas gol yang berlaku masa tu (Data hilang). Tapi kalau kau buka apps **Livescore**, kau tetap dapat tahu skor terkini (macam *Retained Message*).

---

**4. Usage of JSON (JavaScript Object Notation)**

- **Definition:** JSON ialah format pertukaran data yang ***lightweight*** dan ***human-readable***. Ia adalah ***language-independent*** yang disokong oleh hampir semua bahasa pengaturcaraan.
- **Function in IoT Projects:**
    - **Structured Data:** Menggunakan ***Key-Value pairs*** (Contoh: {"suhu": 25}) supaya data tersusun dan tak berterabur.
    - **Efficiency:** Saiz teks yang ringkas mengurangkan penggunaan data semasa *transfer*.
- **Contoh Kehidupan Seharian (Analogi Borang Alamat):** Macam kita tulis alamat ikut kotak: Nama: Ahmad, Poskod: 75450. Kita tak tulis dalam bentuk karangan panjang. Dengan label (Key) ni, komputer senang nak cari mana satu nama dan mana satu nombor tanpa keliru.

---

**Nota Rumusan untuk Lecturer:**

- **MQTT** = Lori yang bawa barang (Transport Protocol).
- **JSON** = Barang yang disusun kemas dalam kotak (Data Format).
- **Broker** = Pejabat Pos yang agihkan barang ikut alamat (Distribution Centre).